

P1 - MC832

Cap. 01 - Introdução

- Internet ^{+ host}
 - ▷ dispositivos conectados
 - ▷ Packet switches: enviam pacotes
 - ↳ roteador, switches
 - ▷ Communication links
 - ▷ Network
 - ▷ Protocols

- Protocols: definem o formato, a ordem das mensagens e ações

- Network edge

- ▷ hosts, servers em datacenters

- ▷ Access networks ^{→ conexões} $\rightarrow$ c/ fio $\vee$ sem fio

DSL $\rightarrow$ cabos HFC: hybrid fiber coax

$\rightarrow$ DSL (Digital subscriber line): usa linha telefônica
DSLAM na central (multiplexers)

$\rightarrow$ home networks / enterprise network / datacenters

$\rightarrow$ Wireless $\left\{ \begin{array}{l} \text{local: Wifi} \\ \text{Wide-area: 3G/4G} \end{array} \right.$

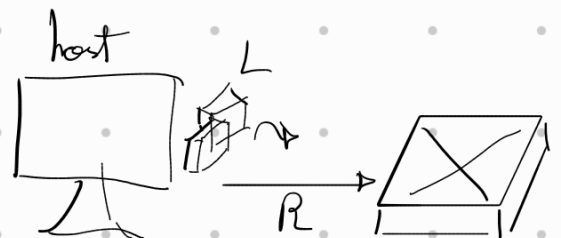
Assimétricas
UPSTREAM
 $\neq$
DOWNSTREAM

- ▷ Host $\rightarrow$ manda pacotes de dados

1º) pega mensagem

2º) quebra em pacotes de comprimento L

3º) transmite os pacotes à uma taxa de transmissão R



$$\text{delay Transmissão} = \text{tempo transmitir } L\text{-bits pelo link} = \frac{L}{R}$$

$L \text{ bits} \cdot \frac{1}{R \text{ bits}}$

- Network core $\rightarrow$ roteadores interconectados

▷ comutação por pacote
(packet-switching)

} host manda pacotes
rede passa os pacotes de um roteador p/ outro
forward

↳ Switching
(local) $\rightarrow$ desloca o pacote de entrada do roteador para a saída apropriada (com base no header)

↳ Routing
(global) $\rightarrow$ determina o caminho entre a origem e destino (atualiza cada forwarding table)

↳ Store-and-Foward $\rightarrow$ o pacote inteiro deve chegar no roteador antes de ser transmitido p/ próxima linha

↳ Queuing: se a arrival rate ultrapassar a transmission rate:

- fila de pacotes
- se o buffer estourar: perda de pacotes

▷ Comutação por circuito
(circuit-switching) $\rightarrow$ aloca recursos de ponta-a-ponta p/ transmissão direta, sem fila ou perda de pacotes. Delay apenas da própria rede.

- $\rightarrow$ sem sharing: fica inativo se não usado.
- $\rightarrow$ Multiplexação por divisão de frequência (FDM) e por divisão de tempo (TDM)

▷ Pacote x Circuito

1 Gb/s link

cada usuário $\rightarrow$ 100 Mb/s quando ativos
(10% tempo)

Quanto usuários simultâneos?

• comutação circuito: $\frac{1000 \text{ Mb/s}}{100 \text{ Mb/s/user}} = 10 \text{ users}$

• comutação pacote: com 35 users $P(n(\text{users}) > 10) < 0,0004$

↳ Comutação por pacote

• ótimo para dados esparsos

• maior capacidade de usuários

• menor rede

• congestionamento: delay e perda de pacotes

• Estrutura da Internet

▷ Hosts se conectam por acesso à ISPs (Internet Service Providers)

▷ ISPs interconectados

▷ Redes de redes

• Performance

▷ Delay $\rightarrow$ 4 fontes

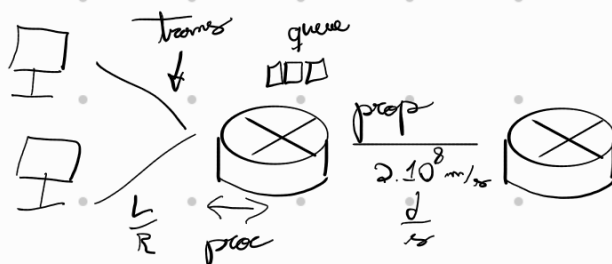
$$d_{\text{total}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

• node processing

• queue

• transmission delay ($d_{\text{trans}} = L/R$)

• propagation delay ($d_{\text{prop}} = d/v$)
 $v = 2 \cdot 10^8 \text{ m/s}$



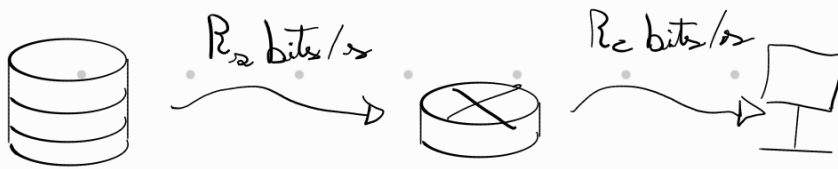
▷ Queuing Delay

- λ : average arrival rate
- L : comprimento pacote
- R : link bandwidth (taxa transmissão)

$L \cdot \lambda =$ arrival rate of bits

$$\frac{\text{intensidade de tráfego}}{\text{tráfego}} = \frac{L \cdot \lambda}{R}$$

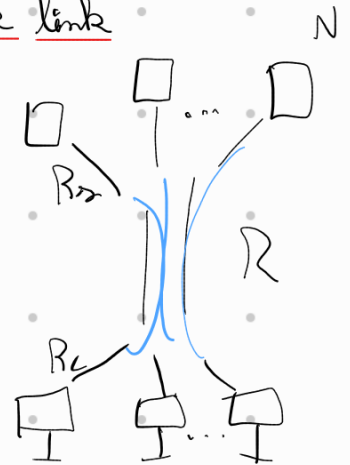
▷ $\tilde{\text{Throughput}}$ → taxa em que os bits estão sendo enviados do emissor p/ receptor
↳ médio x instantâneo



- $R_s < R_c$: $\tilde{\text{Throughput}} = R_s$
- $R_s > R_c$: $\tilde{\text{Throughput}} = R_c$

Bottleneck link

↳ Redes: $\tilde{\text{Throughput}} = \min(R_s, R_c, R/N)$



• Security

- package interception
- fake identity
- DoS

• Protocol Layers

M
 message
 $H_T + M$
 segment
 $H_m + H_T + M$
 datagram
 $H_e + H_m + H_T + M$
 frame



Source

- supporting network applications (HTTP, IMAP, SMTP, DNS)
- process-process data transfer (TCP, UDP)
- routing of datagrams from source to destination (IP, ^{routing} protocols)
- data transfer btw neighboring elements (Ethernet, Wifi, PPP)
- bits on the wire

M
 message
 $H_T + M$
 segment
 $H_m + H_T + M$
 datagram
 $H_e + H_m + H_T + M$
 frame



Destination

Cap. 02 - Camada de Aplicação

- Objetivo: criar programas que rodem em sistemas diferentes e que se comuniquem pela rede

- Paradigmas

- Client-server: } server: sempre ativo, end. IP permanente, datacenters
HTTP client: intermitente, end. IP dinâmico, não comunicam entre si

- Peer-peer: sistemas se comunicam diretamente
P2P intermitente, end. IP dinâmico

- Processo: programa rodando no host

 - ▶ Mesmo host: inter-process communication

 - ▶ Hosts distintos se comunicando: mensagem

 - ▶ Client process: inicia a comunicação

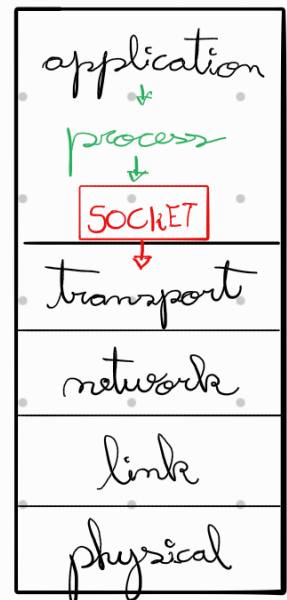
 - ▶ Server process: espera ser contatado

- Socket → processo envia/recebe mensagens pelo socket
→ 1 socket de cada lado

- Endereçamento → host deve ter um identificador

 - ↳ endereço IP

 - ↳ porta



• Serviços de protocolos de transporte

▷ TCP service

- transporte confiável
- controle de fluxo: emissor não sobrecarrega o receptor
- controle de congestionamento: limita o emissor caso a rede esteja sobrecarregada
- orientado à conexão: requer configuração prévia entre o emissor e o receptor
- não prevê timing, throughput mínimo nem segurança

▷ UDP service

- transporte não confiável
- não prevê confiabilidade, controle de fluxo, controle de congestionamento, timing, throughput, segurança ou configuração de conexão

• HTTP → hypertext transfer protocol

▷ protocolo da camada de aplicação da Web

▷ client: browser recebe os objetos da Web

▷ server: Web server manda os objetos em resposta à requisição

▷ usa TCP

- cliente inicia conexão TCP (via sockets) com o server pela port 80
- server aceita conexão
- mensagem HTTP é trocada entre browser e Web server
- conexão TCP é fechada

▷ Stateless: não guarda informações das requisições anteriores do cliente

▷ HTTP não persistente x persistente

não persistente

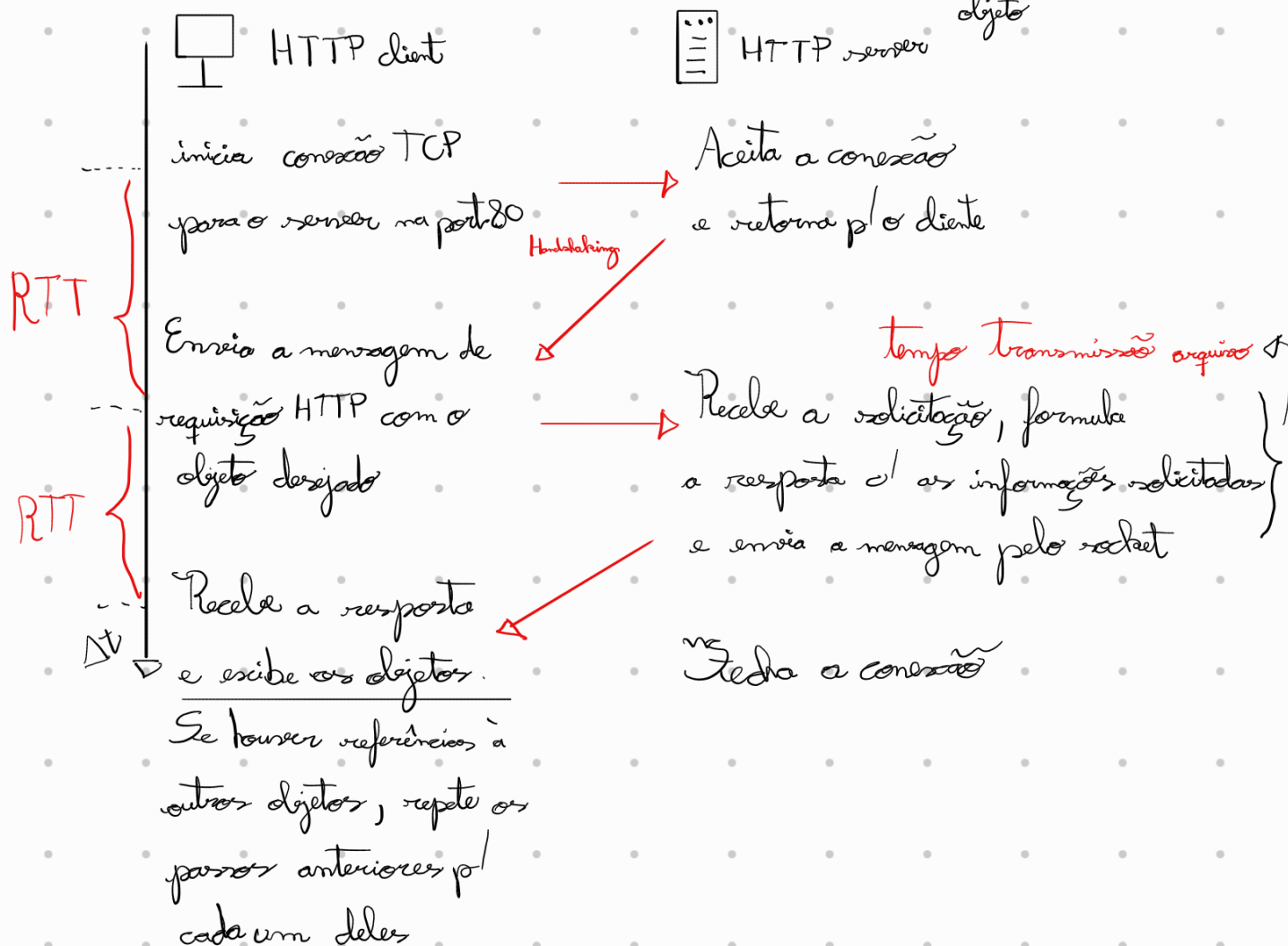
- abre TCP
- envia no máximo 1 objeto
- fecha TCP

persistente (HTTP 1.1)

- abre TCP
- envia múltiplos objetos pela mesma conexão
- fecha TCP

▷ Não persistente

http://www.mysite.com/index.html
objeto



↳ RTT → tempo p/ um pequeno pacote ir e voltar
(round trip time)

$$\text{tempo resposta HTTP não persistente} = 2 \cdot \text{RTT} + \text{tempo transmissão arquivos}$$

↳ problemas

- requer 2 RTT's
- overhead do OS para cada TCP
- browser abre múltiplas conexões TCP em paralelo

▷ Persistente (HTTP 1.1)

- server deixa conexão aberta após mandar a resposta
- novas mensagens são enviadas por este canal
- cliente manda a requisição assim que encontra ref. a um objeto
- menos de 1RTT para todos os objetos referenciados

▷ Mensagens HTTP

- request GET /index.html HTTP/1.1\r\n + header lines
- POST: envio formulários preenchidos pelo usuário
- GET: requisitar uma página ao servidor
- HEAD: requisita apenas os headers
- PUT: upload de um arquivo p/ o server

↗ Status

- response HTTP/1.1 200 OK + header lines

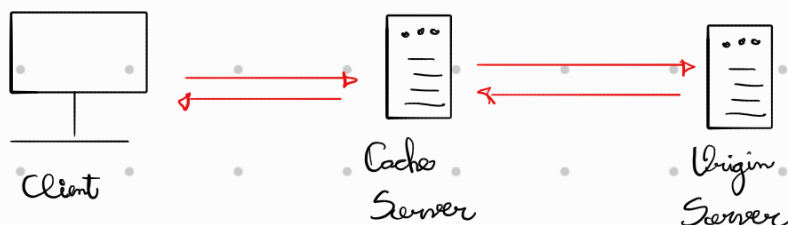
• Cookies

▷ Já que o HTTP é *stateless*, precisamos de uma maneira de armazenar os estados do usuário e os estados entre transações

▷ Componentes

- cookie header line da mensagem de resposta
- cookie header line na próxima mensagem de requisição
- arquivos de cookies na máquina do usuário
- backend database no site Web

• Web caching → atender às requisições do cliente sem invocar o server de origem (proxy servers)



Ex: link 1,54 Mbps $usage = a \cdot L = \frac{15 \cdot 100k}{1000} = 0,97$ / *altre us*

RTT 22 R 1,54 M

direita 100 kbits
 15 req/s

linha x100 mais rápido: $usage = \alpha \frac{L}{R} = \frac{0,97}{100} = 0,0097$

com cache hit de 40%.

$$\alpha = 60\% \pm 15$$
$$u_{\text{age}} = 0,6 \cdot \frac{15 \cdot 100\text{k}}{1,54\text{M}} = 0,582$$
$$\text{delay} = 0,6 \cdot (\text{delay server origen}) + 0,4 \cdot (\text{delay cache})$$

- Email

▷ user agent $\rightarrow$ gmail, outlook, etc.

▷ mail servers → mailbox, message queue

- ▷ protocole SMTP + client : mail server ^{emission}
 - ↳ " "
 - ↳ server : mail server ^{reception}

↳ Use TCP port 25

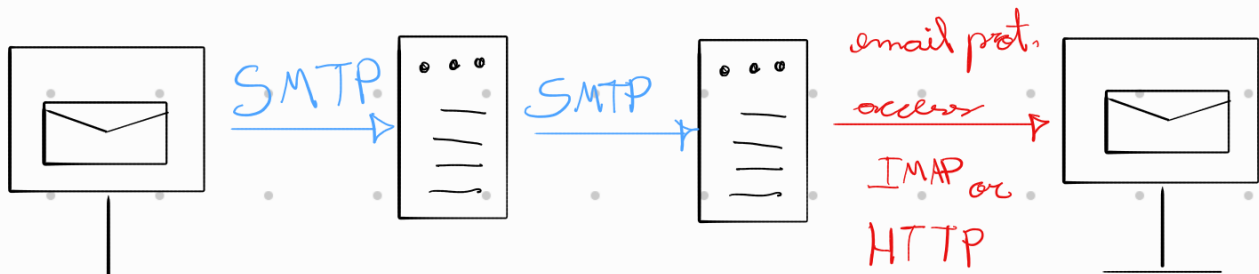
43 faxes

- handshaking

- Transfêrência da mensagem

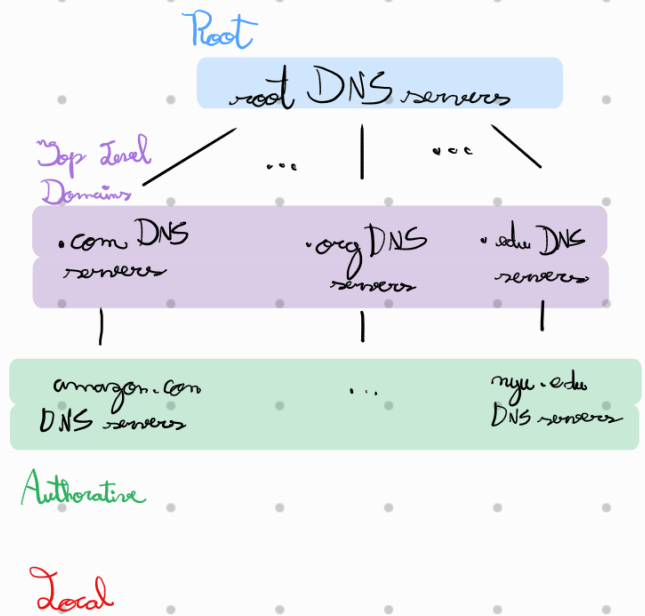
fechamento

↳ HTTP: client pull; SMTP: client push

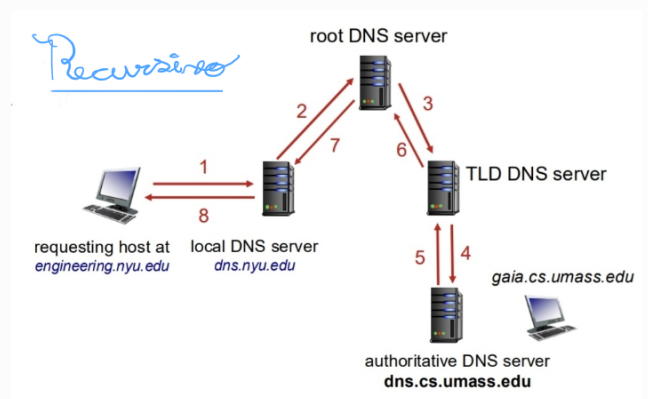
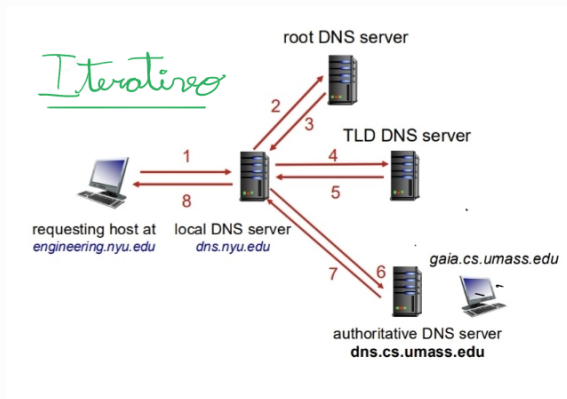


• DNS $\rightarrow$ Domain Name Service

- ▷ mapear end. IP e nome
- ▷ database distribuído com hierarquia de nome servers
- ▷ DNS servers comunicam para resolver nomes
- ▷ DNS services
 - hostname $\rightarrow$ IP address
 - host aliasing / email aliasing
 - load distribution



▷ Iterative x recursive

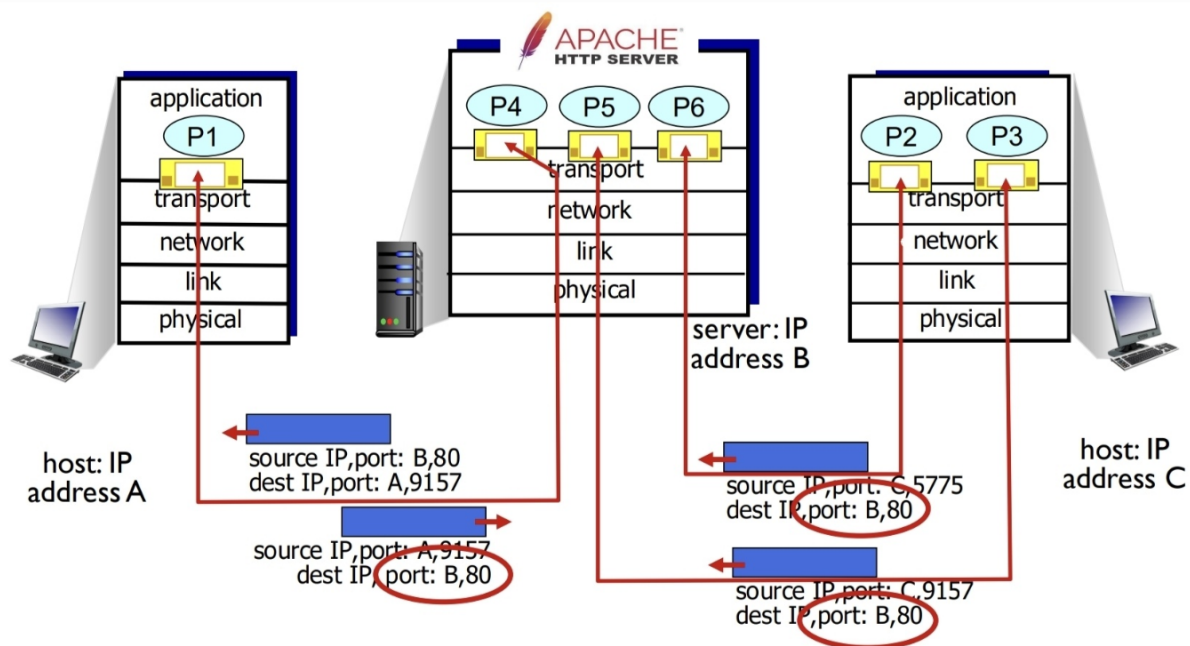


▷ Caching

- ▷ uma vez que o name server aprende o mapeamento, ele guarda na cache
- ▷ expira após TTL (time to live)
- ▷ pode ficar desatualizado

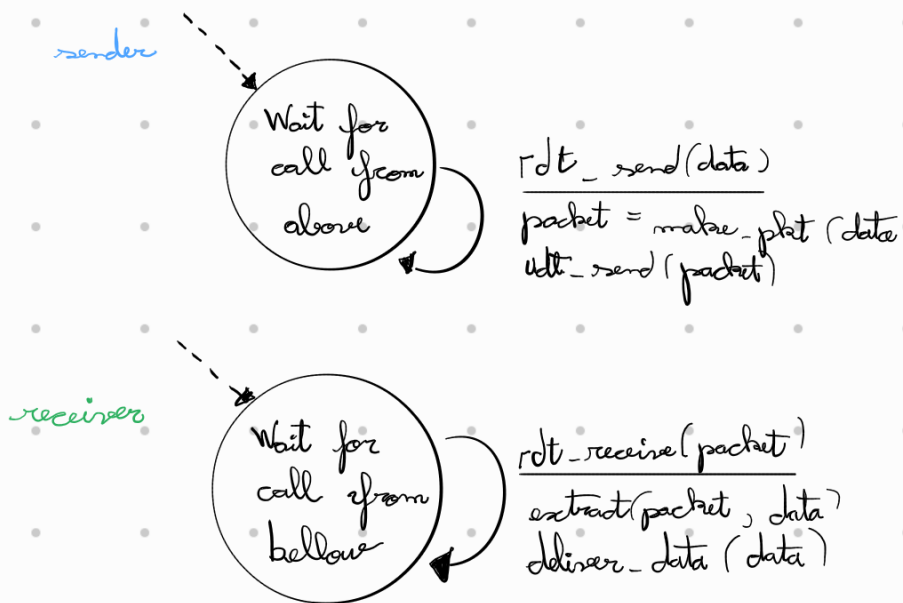
Cap. 03 - Camada de Transporte

- **Objetivo:** prover comunicação lógica entre processos de aplicação rodando em hosts distintos
- **Ações** → Emissor adiciona header, cria segmento e passa p/o IP
→ Receptor recebe a mensagem pelo IP, checa os valores do header, extrai a mensagem e a demultiplexa para a camada de aplicação via socket
- **Multiplexing/Demultiplexing**

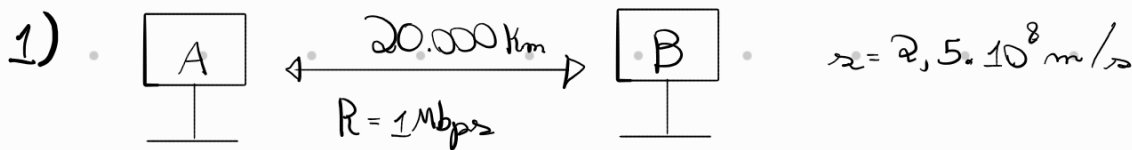


• UDP

- ▷ sem estabelecimento de conexão
- ▷ simples
- ▷ header menor
- ▷ sem controle de congestionamento
- ▷ streaming, DNS, SNMP, HTTP 3.0
- ▷ checksum



Lista 01



a-) $\Delta t = \frac{\Delta S}{c} = \frac{20 \cdot 10^6}{2,5 \cdot 10^8} = 0,08 \text{ s}$

$R \cdot \Delta t = 10^6 \text{ bits} \cdot 0,08 \text{ s} = 80 \text{ kb}$

b-) $L = 200.000 \text{ bits}$ $n_{\text{máx}} = 80.000 \text{ bits}$

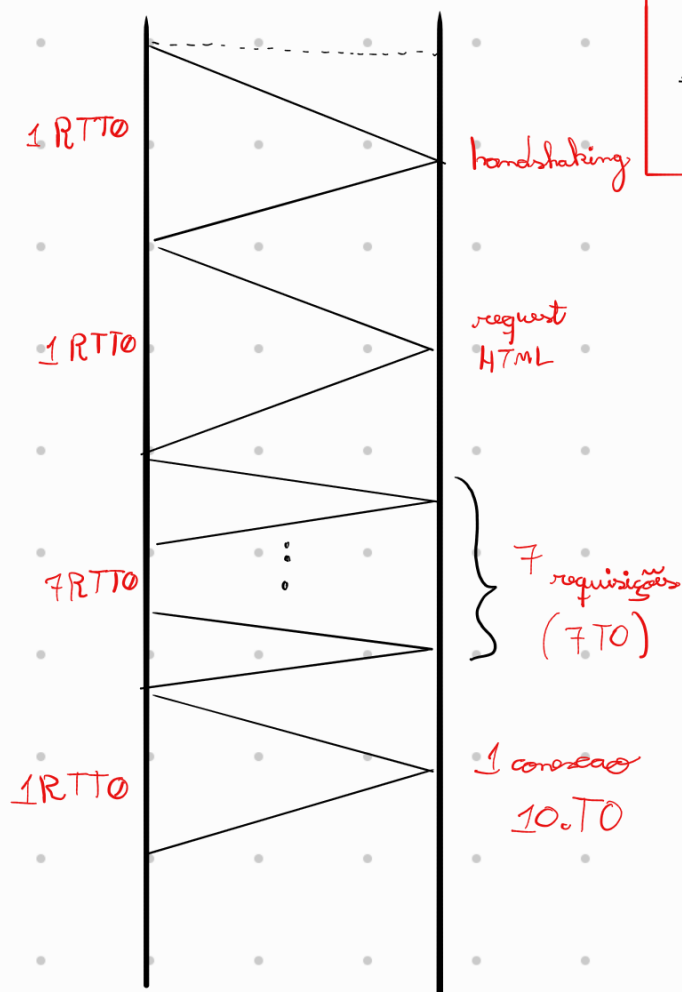
c-) $R \cdot \Delta t$ corresponde a quantidade máxima, em bits, na conexão de enlace simultaneamente.

d-) $\lambda = \frac{80.000 \text{ bits}}{200.000 \text{ m}} = \frac{1 \text{ bit}}{250 \text{ m}}$

2-) O acesso residencial HFC é baseado em um único cabo coaxial, pelo qual é transmitido os dados de todas as residências pelo mesmo meio. O fio se encontra na linha de poste, sendo bifurcado para cada casa. Já a DSL atua através da linha telefônica de cada residência. Neste caso, cabe à central mandar o sinal correto para cada residência por meio de um multiplexador (DSLAM).

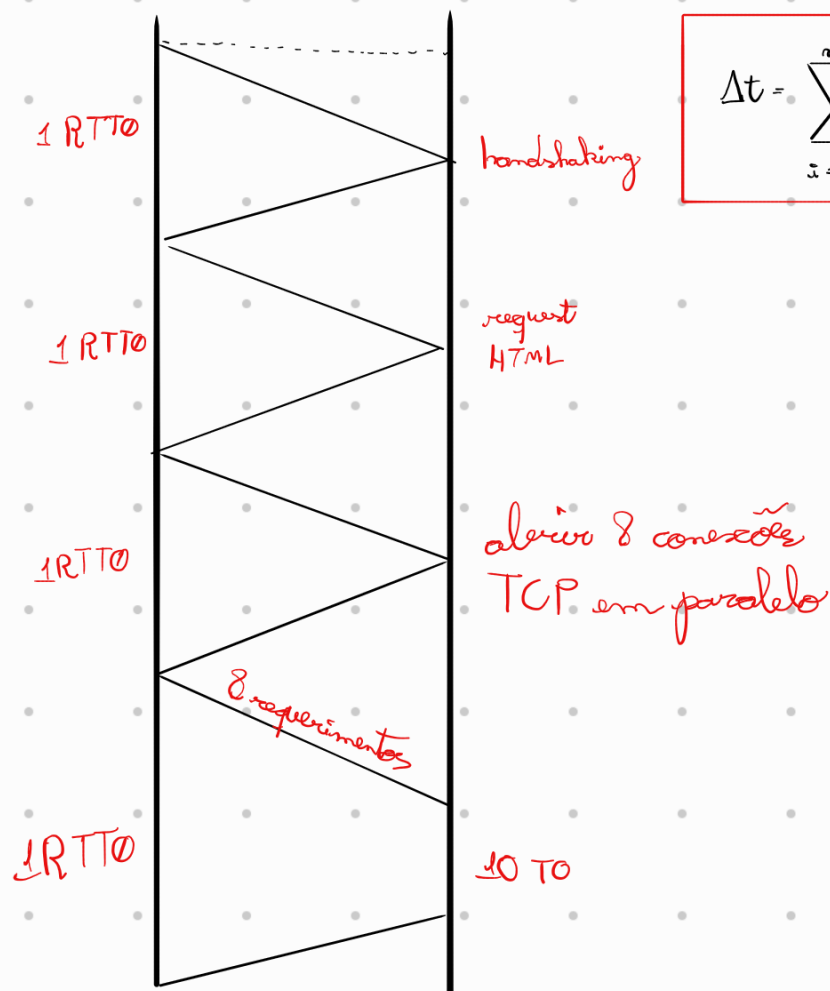
3-) $DNS_{base} = \sum_{i=1}^n RTT_i$

a-)



$$\Delta T = \sum_{i=1}^n (RTT_i) + 10 RTT_0 + 17 T_0$$

b)



$$\Delta t = \sum_{i=1}^n (RTT_i) + 4 RTT_0 + 10 T_0$$

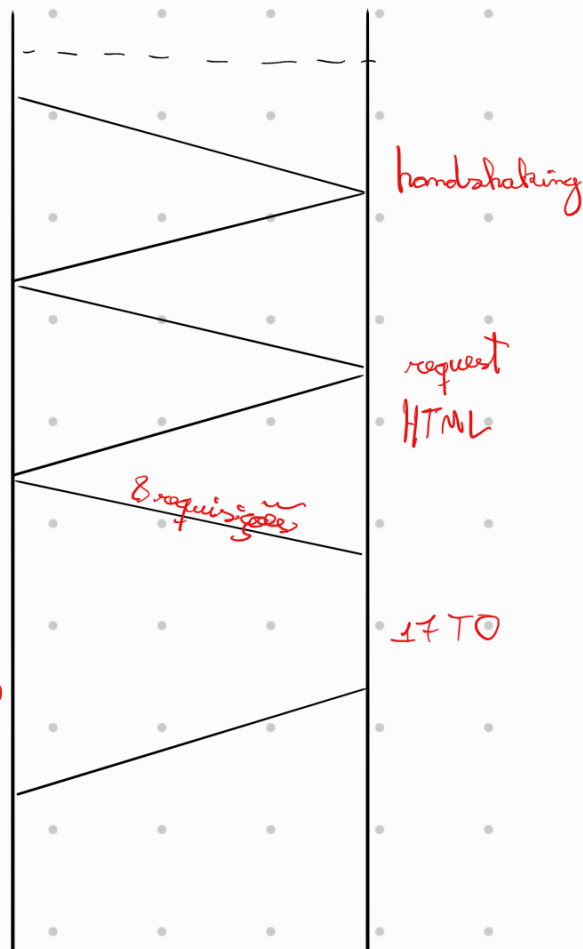
c)

$$\Delta t = \sum_{i=1}^n (RTT_i) + 3 RTT_0 + 17 T_0$$

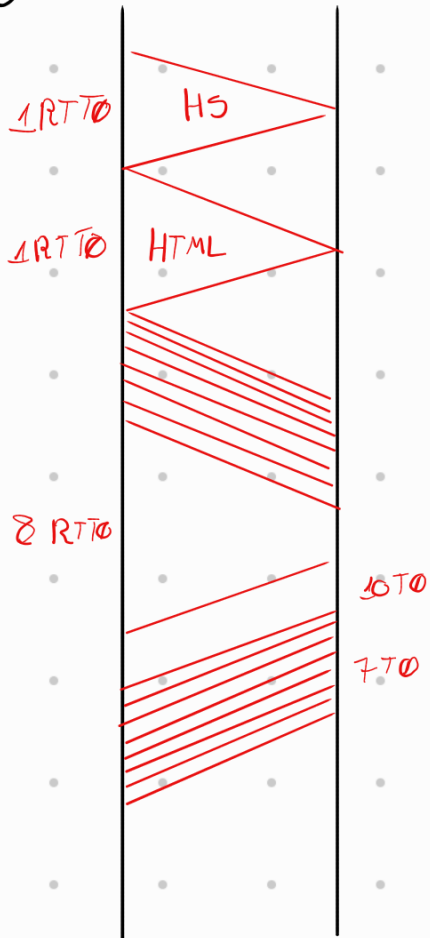
$1 RTT_0$

$1 RTT_0$

$1 RTT_0$



3-) a-)



$$\Delta t = \sum_{j=1}^m (RTT_j) + 10RTT + 17TO$$

$$\Delta t_0 = \sum_{i=1}^m (RTT_i) + 3RTT + 10TO$$

b-)

$$\Delta t = \sum_{j=1}^m RTT_j + 4RTT + 10TO$$

